



使用 KNN 算法对鸢尾花数据集进行分类

下面是完整的实现代码，按照步骤一步步实现：

代码块

```
1  # ===== 步骤 1: 导入所需的库和模块 =====
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  from sklearn.datasets import load_iris
7  from sklearn.model_selection import train_test_split, cross_val_score,
   GridSearchCV
8  from sklearn.preprocessing import StandardScaler
9  from sklearn.neighbors import KNeighborsClassifier
10 from sklearn.metrics import accuracy_score, precision_score, recall_score,
   f1_score, classification_report, confusion_matrix
11 import warnings
12 warnings.filterwarnings('ignore')
13
14 # 设置中文字体显示
15 plt.rcParams['font.sans-serif'] = ['SimHei']
16 plt.rcParams['axes.unicode_minus'] = False
17
18 print("="*60)
19 print("🌸 KNN 尾花分类任务")
20 print("="*60)
21
22 # ===== 步骤 2: 加载鸢尾花数据集 =====
23 print("\n📁 步骤 2: 加载尾花数据集")
24 print("-"*60)
25
26 # 使用 sklearn 内置的 load_iris 函数加载数据
27 iris = load_iris()
28 X = iris.data      # 特征数据
29 y = iris.target    # 目标变量 (类别标签)
30
31 # 显示数据集信息
32 print(f"数据集样本数量: {X.shape[0]}")
33 print(f"特征数量: {X.shape[1]}")
34 print(f"特征名称: {iris.feature_names}")
```

```

35 print(f"类别名称: {iris.target_names}")
36 print(f"每个类别的样本数: ")
37 for i, name in enumerate(iris.target_names):
38     print(f"    - {name}: {np.sum(y == i)} 个样本")
39
40 # ===== 步骤 3: 将数据集分为特征和目标变量 =====
41 print(f"\n📊 步骤 3: 划分特征和目标变量")
42 print("-"*60)
43
44 # X 已经是特征矩阵, y 已经是目标变量
45 print(f"特征矩阵 X 形状: {X.shape}")
46 print(f"目标变量 y 形状: {y.shape}")
47
48 # 显示前 5 个样本
49 print(f"\n前 5 个样本的特征: ")
50 print(X[:5])
51 print(f"\n前 5 个样本的类别标签: ")
52 print(y[:5])
53 print(f"对应的类别名称: {[iris.target_names[label] for label in y[:5]]}")
54
55 # ===== 步骤 4: 对特征进行标准化处理 =====
56 print(f"\n📏 步骤 4: 特征标准化")
57 print("-"*60)
58
59 # 创建标准化器
60 scaler = StandardScaler()
61
62 # 拟合并转换数据
63 X_scaled = scaler.fit_transform(X)
64
65 print(f"✅ 标准化完成! ")
66 print(f"\n标准化前 (前 3 个样本): ")
67 print(X[:3])
68 print(f"\n标准化后 (前 3 个样本): ")
69 print(X_scaled[:3])
70 print(f"\n标准化后特征的均值: {X_scaled.mean(axis=0)}")
71 print(f"\n标准化后特征的标准差: {X_scaled.std(axis=0)}")
72
73 # ===== 步骤 5: 将数据集划分为训练集和测试集 =====
74 print(f"\n✂️ 步骤 5: 划分训练集和测试集")
75 print("-"*60)
76
77 # 划分数数据集 (80% 训练, 20% 测试)
78 X_train, X_test, y_train, y_test = train_test_split(
79     X_scaled,          # 标准化后的特征
80     y,                 # 目标变量
81     test_size=0.2,     # 测试集比例 20%

```

```

82     random_state=42,      # 随机种子 (保证可重复性)
83     stratify=y            # 分层抽样 (保持各类别比例)
84 )
85
86 print(f"训练集大小: {X_train.shape[0]} 个样本")
87 print(f"测试集大小: {X_test.shape[0]} 个样本")
88 print(f"\n训练集各类别分布: ")
89 for i, name in enumerate(iris.target_names):
90     count = np.sum(y_train == i)
91     print(f" - {name}: {count} 个样本 ({count/len(y_train)*100:.1f}%)")
92
93 # ===== 步骤 6: 使用 KNN 算法训练模型 =====
94 print("\n🧠 步骤 6: 训练 KNN 分类模型")
95 print("-"*60)
96
97 # 创建 KNN 分类器
98 # n_neighbors: K 值, 通常选择奇数避免平票
99 # weights: 权重方式, 'uniform'表示所有邻居权重相同
100 # metric: 距离度量方式, 'euclidean'表示欧氏距离
101 knn_classifier = KNeighborsClassifier(
102     n_neighbors=5,          # K 值
103     weights='uniform',     # 权重: 'uniform'或'distance'
104     metric='euclidean',    # 距离度量
105     n_jobs=-1              # 使用所有 CPU 核心
106 )
107
108 # 训练模型 (KNN 是懒惰学习, 实际只是存储数据)
109 knn_classifier.fit(X_train, y_train)
110 print("✅ 模型训练完成!")
111
112 # ===== 步骤 7: 使用测试集对模型进行评估 =====
113 print("\n📊 步骤 7: 模型评估")
114 print("-"*60)
115
116 # 在测试集上进行预测
117 y_pred = knn_classifier.predict(X_test)
118
119 # 计算各项评估指标
120 accuracy = accuracy_score(y_test, y_pred)
121 precision = precision_score(y_test, y_pred, average='weighted')
122 recall = recall_score(y_test, y_pred, average='weighted')
123 f1 = f1_score(y_test, y_pred, average='weighted')
124
125 print("📊 模型性能指标: ")
126 print(f"{'='*40}")
127 print(f"准确率 (Accuracy) : {accuracy:.4f} ({accuracy*100:.2f}%)")
128 print(f"精确率 (Precision) : {precision:.4f} ({precision*100:.2f}%)")

```

```

129 print(f"召回率 (Recall)      : {recall:.4f} ({recall*100:.2f}%)")
130 print(f"F1 分数 (F1-Score) : {f1:.4f} ({f1*100:.2f}%)")
131 print(f"{'='*40}")
132
133 # 显示详细分类报告
134 print(f"\n 详细分类报告:")
135 print(classification_report(y_test, y_pred, target_names=iris.target_names))
136
137 # 显示混淆矩阵
138 print(f" 混淆矩阵:")
139 cm = confusion_matrix(y_test, y_pred)
140 print(cm)
141
142 # ===== 步骤 8: 使用模型预测未知样本的类别 =====
143 print(f"\n🧠 步骤 8: 预测未知样本")
144 print(f"{'-'*60}")
145
146 # 创建一些未知的鸢尾花样本数据 (萼片长度、萼片宽度、花瓣长度、花瓣宽度)
147 unknown_samples = np.array([
148     [5.1, 3.5, 1.4, 0.2], # 可能是山鸢尾
149     [6.2, 3.4, 5.4, 2.3], # 可能是维吉尼亚鸢尾
150     [5.9, 3.0, 4.2, 1.5], # 可能是变色鸢尾
151     [4.8, 3.0, 1.4, 0.3], # 可能是山鸢尾
152     [7.0, 3.2, 4.7, 1.4]  # 可能是变色鸢尾
153 ])
154
155 # 对未知样本进行标准化 (使用训练集的标准器)
156 unknown_samples_scaled = scaler.transform(unknown_samples)
157
158 # 预测类别
159 predicted_classes = knn_classifier.predict(unknown_samples_scaled)
160 predicted_probabilities = knn_classifier.predict_proba(unknown_samples_scaled)
161
162 print(f"🌸 未知样本预测结果:")
163 print(f"{'='*60}")
164 print(f"{'样本':<6} {'萼片长':<8} {'萼片宽':<8} {'花瓣长':<8} {'花瓣宽':<8} {'预测类别':<15} {'置信度':<10}")
165 print(f"{'-'*60}")
166
167 for i, (sample, pred_class, probs) in enumerate(zip(unknown_samples,
predicted_classes, predicted_probabilities)):
168     confidence = probs[pred_class]
169     pred_name = iris.target_names[pred_class]
170     print(f"{i+1:<8} {sample[0]:<10.1f} {sample[1]:<10.1f} {sample[2]:<10.1f}
{sample[3]:<10.1f} {pred_name:<18} {confidence*100:.2f}%")
171
172 print(f"{'='*60}")

```

```

173
174 # ===== 附加: K 值优化 =====
175 print("\n🔍 附加: K 值优化 (交叉验证) ")
176 print("-"*60)
177
178 # 测试不同的 K 值
179 k_range = range(1, 21)
180 cv_scores = []
181
182 for k in k_range:
183     knn = KNeighborsClassifier(n_neighbors=k)
184     scores = cross_val_score(knn, X_scaled, y, cv=5, scoring='accuracy')
185     cv_scores.append(scores.mean())
186
187 # 找到最佳 K 值
188 best_k = k_range[np.argmax(cv_scores)]
189 best_score = max(cv_scores)
190
191 print(f"不同 K 值的交叉验证准确率: ")
192 print(f"{'K 值':<10} {'准确率':<10}")
193 print(f"{'-'*20}")
194 for k, score in zip(k_range, cv_scores):
195     marker = " ← 最佳" if k == best_k else ""
196     print(f"{'k':<12} {'score':.4f}{marker}")
197
198 print(f"\n🌟 最佳 K 值: {best_k}, 准确率: {best_score:.4f}
199 ({best_score*100:.2f}%)")
200
201 # 使用最佳 K 值重新训练模型
202 best_knn = KNeighborsClassifier(n_neighbors=best_k)
203 best_knn.fit(X_train, y_train)
204 y_pred_best = best_knn.predict(X_test)
205
206 print(f"\n使用最佳 K 值 ({best_k}) 的模型在测试集上的准确率: ")
207 print(f"准确率: {accuracy_score(y_test, y_pred_best):.4f}
208 ({accuracy_score(y_test, y_pred_best)*100:.2f}%)")
209
210 # ===== 可视化 =====
211 print("\n📊 可视化结果")
212 print("-"*60)
213
214 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
215
216 # 图 1: K 值与准确率的关系
217 axes[0].plot(k_range, cv_scores, marker='o', linewidth=2, markersize=8,
218 color='blue')
219 axes[0].set_xlabel('K Value', fontsize=12)

```

```

217 axes[0].set_ylabel('Accuracy', fontsize=12)
218 axes[0].set_title('K Value vs Accuracy', fontsize=14)
219 axes[0].grid(True, alpha=0.3)
220 axes[0].annotate(f'Best K={best_k}\nAccuracy={best_score:.4f}',
221                  xy=(best_k, best_score),
222                  xytext=(best_k+2, best_score-0.05),
223                  arrowprops=dict(arrowstyle='->', color='red'),
224                  fontsize=10, color='red',
225                  bbox=dict(boxstyle='round,pad=0.3', facecolor='yellow',
alpha=0.5))
226
227 # 图 2: 混淆矩阵热力图
228 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
229             xticklabels=iris.target_names,
230             yticklabels=iris.target_names,
231             ax=axes[1])
232 axes[1].set_xlabel('Predicted', fontsize=12)
233 axes[1].set_ylabel('True', fontsize=12)
234 axes[1].set_title('Confusion Matrix', fontsize=14)
235
236 plt.tight_layout()
237 plt.show()
238
239 print("\n" + "="*60)
240 print("✅ 任务完成! ")
241 print("="*60)

```

代码说明

核心步骤解析:

1. 步骤 1 - 导入库

- 导入必要的机器学习库和工具

2. 步骤 2 - 加载数据

- 使用 `load_iris()` 加载鸢尾花数据集
- 数据集包含 150 个样本, 4 个特征, 3 个类别

3. 步骤 3 - 划分特征和目标

- X: 特征矩阵 (150×4)
- y: 类别标签 (150,)

4. 步骤 4 - 标准化

- 使用 `StandardScaler` 将特征标准化为均值为 0，方差为 1
- **重要：**KNN 对特征尺度非常敏感，必须标准化

5. 步骤 5 - 划分数据集

- 80% 训练集，20% 测试集
- 使用 `stratify=y` 保持各类别比例一致

6. 步骤 6 - 训练模型

- 使用 `KNeighborsClassifier`
- KNN 是懒惰学习，训练时只存储数据

7. 步骤 7 - 模型评估

- 计算准确率、精确率、召回率、F1 分数
- 显示分类报告和混淆矩阵

8. 步骤 8 - 预测未知样本

- 对新样本进行标准化
- 预测类别并显示置信度

关键点：

- **标准化必须做：**KNN 基于距离计算，特征尺度不同会导致偏差
- **K 值选择：**使用交叉验证选择最佳 K 值
- **分层抽样：** `stratify=y` 确保训练集和测试集类别分布一致

- 懒惰学习：KNN 训练很快，但预测时需要计算所有距离

输出结果示例：

代码块

```
1  未知样本预测结果：
2  =====
3  样本      萼片长      萼片宽      花瓣长      花瓣宽      预测类别      置信度
4  -----
5  1          5.1          3.5          1.4          0.2          setosa          100.00%
6  2          6.2          3.4          5.4          2.3          virginica       100.00%
7  3          5.9          3.0          4.2          1.5          versicolor     80.00%
8  4          4.8          3.0          1.4          0.3          setosa          100.00%
9  5          7.0          3.2          4.7          1.4          versicolor     100.00%
10 =====
```

运行这段代码即可完成完整的 KNN 鸢尾花分类任务！🌸✨